B26-L6 lab: Compare fine-tuning strategies, then fine-tune a model on your own task

Platform: Laptop

Work in `labs/finetune/` in this repository. You pretrain a small transformer on unlabelled sequences, then fine-tune it on 60 labels four ways: from scratch, a frozen encoder with a new head, full fine-tuning and LoRA. You write the weight loading, the freezing, the warm-up schedule and LoRA, and the tests check each. You need PyTorch 2.15 on the laptop. Then apply the same steps with Hugging Face to a work task. That fine-tuned model is the roadmap's evidence for month 12.

1. Run `cd labs/finetune`, create a virtual environment, then `pip install -e .` and `pip install pytest`. Run `pytest`: most tests fail with NotImplementedError. Read `README.md` and every file in `src/finetune_lab/`.
2. Write `warmup_linear` in `schedule.py` (lesson B26-L2). Run `pytest tests/test_data_and_schedule.py`.
3. Write `load_backbone`, `set_trainable` and `count_parameters` in `strategies.py` (lesson B26-L1), then `LoRALinear` and `add_lora` in `lora.py` (lesson B26-L3). Run `pytest`: every test passes.
4. Run `python -m finetune_lab pretrain`, then `python -m finetune_lab compare` and `python -m finetune_lab compare --n-train 600`. Compare your tables with the README's reference numbers.
5. Choose a work task with a few hundred labelled texts: coded open answers for classification, or brand names for tagging. Fine-tune a small pretrained encoder from the Hugging Face Hub with transformers and peft (lessons B26-L4 and B26-L5). Use integer labels and `task_type`, and check the load report.

6. Evaluate as in this lesson: a majority-class baseline, a simple model, LoRA against full fine-tuning, three seeds, entity-level F1 if you tag. Score the test set once.
7. Write your README: the lab tables, your task, the data, every score with its range, and what you would try next. Push it to a new GitHub repository.

Notes